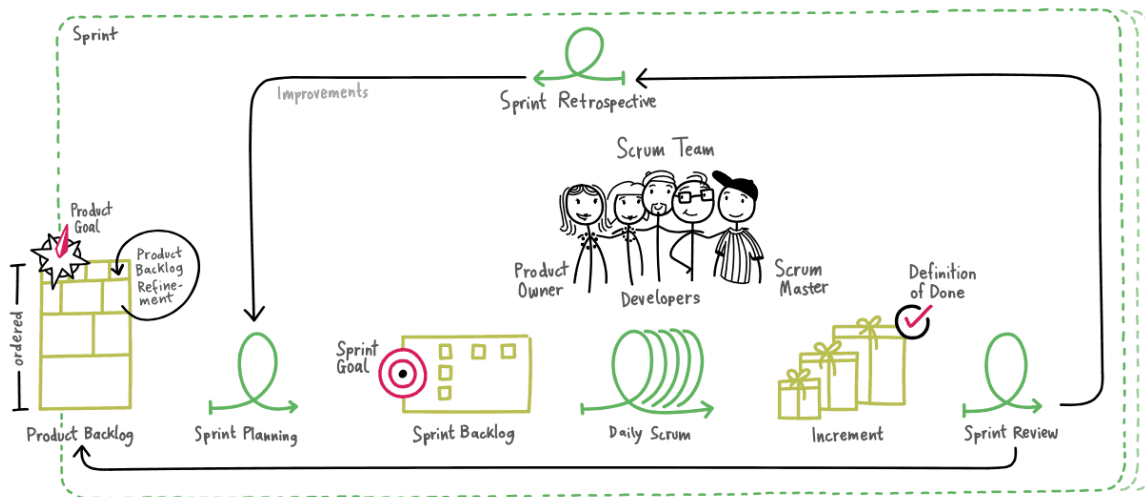


Introduction to Scrum Framework

Scrum is a framework designed to deliver software products that comprise three roles, three artifacts, and Five events.

These basic elements provide a structure to manage the decisions, risks, actions, and uncertainties that come with building and maintaining a software product. Scrum is deliberately incomplete, in that it goes no further than specifying who is accountable for what, with a basic structure to ask the right questions. Once established, as [Scrum](#) team will complete the process by adding the business- and technology-specific processes that make sense for their work. Over time, a mature team will evolve a bespoke process based around the core Scrum framework that is as efficient and effective as their skills and imaginations allow.

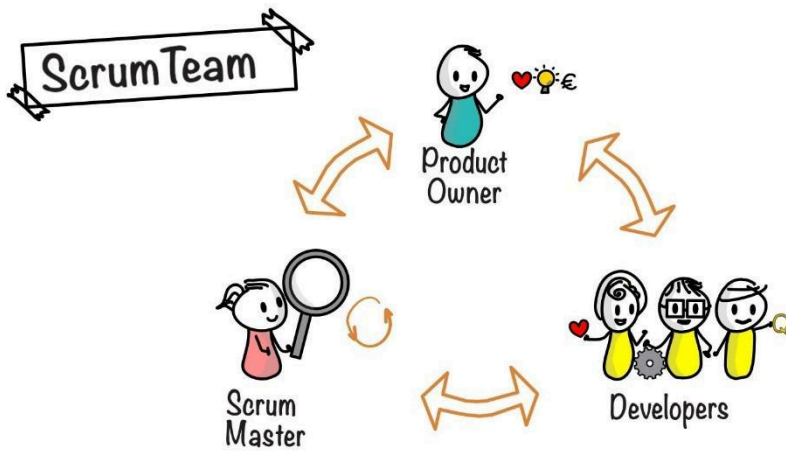
Scrum Framework



Scrum Roles

The fundamental unit of Scrum is a small team of people, a *Scrum Team* that consists of one [Scrum Master](#), one Product Owner, and Developers. Within a Scrum Team, there are no sub-teams or hierarchies. It is a cohesive unit of professionals focused on one objective at a time, the Product Goal.

- Scrum Teams are cross-functional which implies that the members have all the skills necessary to create value for each Sprint.
- They are self-managing, meaning they internally decide who does what, when, and how.
- The team is responsible for all product-related activities from stakeholder collaboration, verification, maintenance, operation, experimentation, research and development, and anything else that might be required.
- They are structured and empowered by the organization to manage their own work.
- Working in Sprints at a sustainable pace improves the Scrum Team's focus and consistency.



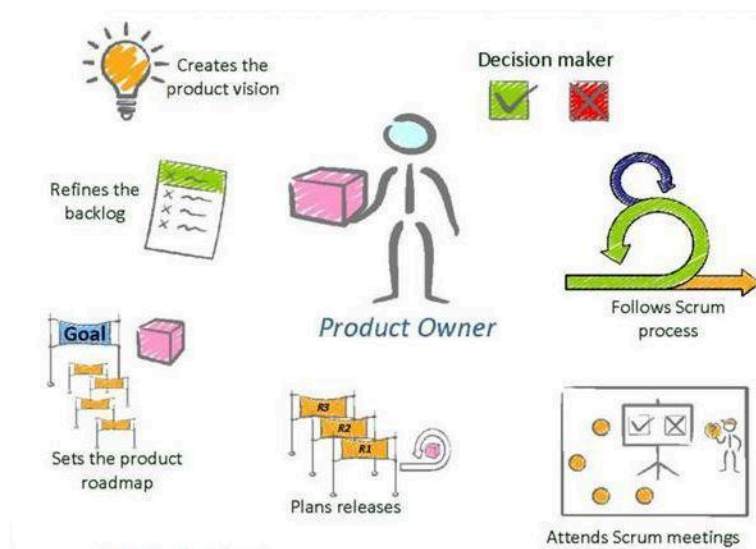
1.) Product Owner

The Product Owner is accountable for maximizing the value of the product resulting from the work of the Scrum Team for which the method may vary widely across organizations, Scrum Teams, and individuals.

The Product Owner is also accountable for effective Product Backlog management, including:

- Developing and explicitly communicating the Product Goal
- Creating and clearly communicating Product Backlog items
- Ordering Product Backlog items
- Ensuring that the Product Backlog is transparent, visible, and understandable

The Product Owner may do the above work or delegate the responsibility to others but in either case, the product owner is accountable for it.



2.) Scrum Master – what is part of the role of the scrum master

The Scrum Master is accountable for establishing Scrum by helping everyone understand Scrum theory and practice, both within the Scrum Team and the organization. He is accountable for the Scrum Team's effectiveness. They do this by enabling the Scrum Team to improve its practices, within the Scrum framework. Scrum Masters are true leaders who serve the Scrum Team and the larger organization.

The Scrum Master serves the Scrum Team in several ways, including:

- Coaching the team members in self-management and cross-functionality;
- Helping the Scrum Team focus on creating high-value Increments that meet the Definition of Done;

- Causing the removal of impediments to the Scrum Team's progress; and,
- Ensuring that all Scrum events take place and are positive, productive, and kept within the timebox.

The Scrum Master also serves the Product Owner in several ways, including:

- Help in finding techniques for effective Product Goal definition and Product Backlog management
- Help the Scrum Team understand the need for clear and concise Product Backlog items
- Help establish empirical product planning for a complex environment
- Facilitate stakeholder collaboration as requested or required

The Scrum Master serves the organization in the following ways:

- Leading, training, and coaching the organization in its Scrum adoption
- Planning and advising Scrum implementations within the organization
- Help employees and stakeholders understand and enact an empirical approach for complex work
- Removing barriers between stakeholders and Scrum Teams



3.) Developers

Developers are the people in the Scrum Team that are committed to creating any aspect of a usable Increment for each Sprint.

The specific skills needed by the Developers are often broad and vary with the domain of work. However, the Developers are always accountable for:

- Creating a plan for the Sprint, the Sprint Backlog
- Instilling quality by adhering to a Definition of Done (DoD)
- Adapting their plan each day toward the Sprint Goal
- Holding each other accountable as professionals



Scrum Events

The Sprint is a container for all other events. Each event in Scrum is a formal opportunity to inspect and adapt artifacts scrum. These events are specifically designed to enable the transparency required. Failure to operate any events as prescribed results in lost opportunities to inspect and adapt. Events are used in Scrum to create regularity and to minimize the need for meetings not defined in Scrum.

Optimally, all events are held at the same time and place to reduce complexity.

- The Sprint
- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

1.) The Sprint

Sprints are the heartbeat of Scrum, where ideas are turned into value. These are fixed length events of one month or less to create consistency. A new Sprint starts immediately after the conclusion of the previous Sprint. All the work necessary to achieve the Product Goal, including Sprint Planning, Daily Scrums, Sprint Review, and Sprint Retrospective, happen within Sprints. A Sprint could be canceled if the Sprint Goal becomes obsolete and only the Product Owner has the authority to cancel the Sprint.

2.) Sprint Planning

Sprint Planning initiates the Sprint by laying out the work to be performed for the Sprint. This resulting plan is created by the collaborative work of the entire Scrum Team. The Product Owner ensures that attendees are prepared to discuss the most important Product Backlog items and how they map to the Product Goal. The Scrum Team may also invite other people to attend Sprint Planning to provide advice. Sprint Planning is timeboxed to a maximum of eight hours for a one-month Sprint. For shorter Sprints, the event is usually shorter.

In the Sprint Planning the following topics are addressed:

- Why is this Sprint valuable?
- What can be done this Sprint?
- How will the chosen work get done?

3.) Daily Scrum

The purpose of the Daily Scrum is to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary, adjusting the upcoming planned work. The Daily Scrum is a 15-minute event for the Developers of the Scrum Team. To reduce complexity, it is held at the same time and place every working day of the Sprint. If the Product Owner or Scrum Master are actively working on items in the Sprint Backlog, they participate as Developers. Daily Scrums improve communications, identify impediments, promote quick decision-making, and consequently eliminate the need for other meetings.

4.) Sprint Review

The purpose of the Sprint Review is to inspect the outcome of the Sprint and determine future adaptations. The Scrum Team presents the results of their work to key stakeholders and progress toward the Product Goal is discussed.

During the event, the Scrum Team and stakeholders review what was accomplished in the Sprint and what has changed in their environment. Based on this information, attendees collaborate on what to do next. The Product Backlog may also be adjusted to meet new opportunities. The Sprint Review is a working session and the Scrum Team should avoid limiting it to a presentation.

It is the second to last event of the Sprint and is timeboxed to a maximum of four hours for a one-month Sprint.

5.) Sprint Retrospective

The purpose of the Sprint Retrospective is to plan ways to increase quality and effectiveness.

The Scrum Team inspects how the last Sprint went with regards to individuals, interactions, processes, tools, and their Definition of Done. Inspected elements often vary with the domain of work. Assumptions that led them astray are identified and their origins explored. The Scrum Team discusses what went well during the Sprint, what problems it encountered, and how those problems were (or were not) solved.

The Scrum Team identifies the most helpful changes to improve its effectiveness. The most impactful improvements are addressed as soon as possible. They may even be added to the Sprint Backlog for the next Sprint.

The Sprint Retrospective concludes the Sprint. It is timeboxed to a maximum of three hours for a one-month Sprint. For shorter Sprints, the event is usually shorter.

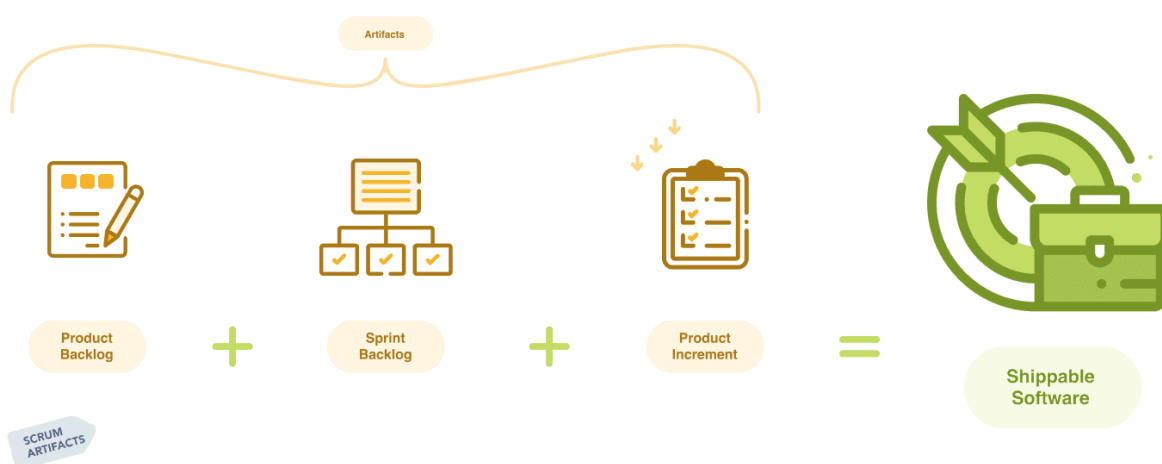
Scrum Artifacts

Scrum's artifacts represent work or value. They are designed to maximize the transparency of key information so that everyone inspecting them has the same basis for adaptation.

Each artifact contains a commitment to ensure it provides information that enhances transparency and focus against which progress can be measured:

- For the Product Backlog, it is the Product Goal.
- For the Sprint Backlog, it is the Sprint Goal.
- For the Increment, it is the Definition of Done.

These commitments exist to reinforce empiricism and the Scrum values for the Scrum Team and its stakeholders.



Scrum Artifacts List

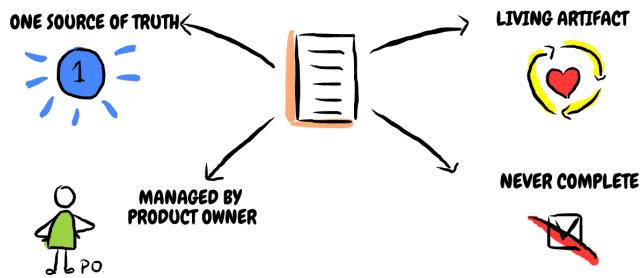
1.) Product Backlog

The Product Backlog is an emergent, ordered list of what is needed to improve the product. It is the single source of work undertaken by the Scrum Team.

Product Backlog items that can be done by the Scrum Team within one Sprint are deemed ready for selection in a Sprint Planning event. They usually acquire this degree of transparency after refining activities. Product Backlog refinement is the act of breaking down and further defining Product Backlog items into smaller more precise

items. This is an ongoing activity to add details, such as a description, order, and size. Attributes often vary with the domain of work.

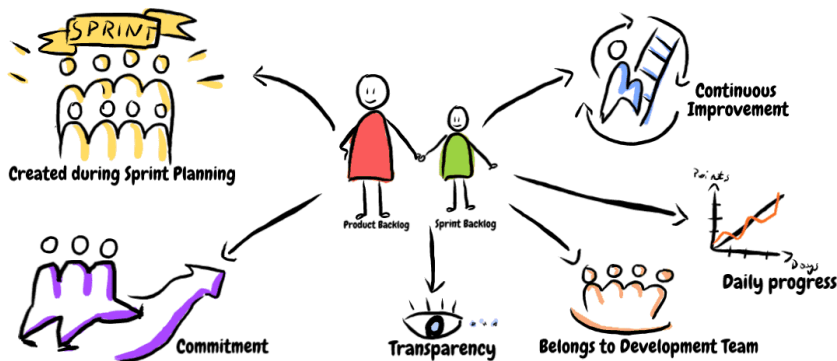
The Developers who will be doing the work are responsible for the sizing. The Product Owner may influence the Developers by helping them understand and select trade-offs.



2.) Sprint Backlog

The Sprint Backlog is composed of the Sprint Goal (why), the set of Product Backlog items selected for the Sprint (what), as well as an actionable plan for delivering the Increment (how).

The Sprint Backlog is a plan by and for the Developers. It is a highly visible, real-time picture of the work that the Developers plan to accomplish during the Sprint in order to achieve the Sprint Goal. Consequently, the Sprint Backlog is updated throughout the Sprint as more is learned. It should have enough detail that they can inspect their progress in the Daily Scrum.

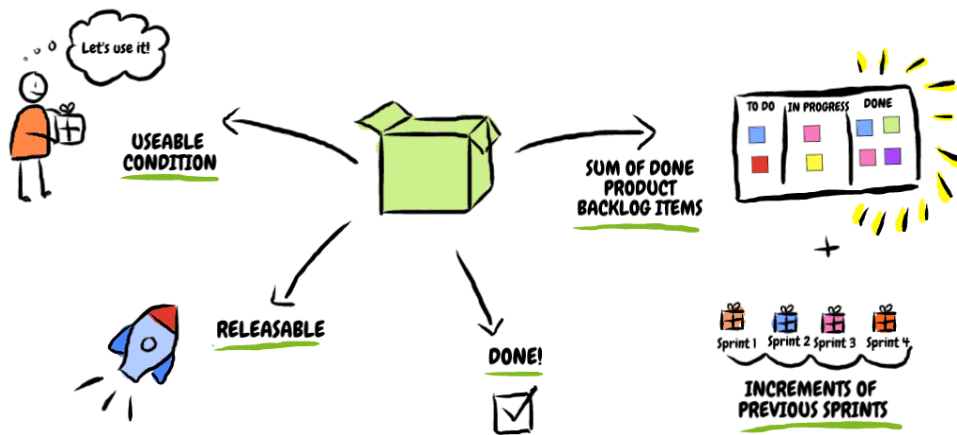


3.) Increment

An Increment is a concrete stepping stone toward the Product Goal. Each Increment is additive to all prior Increments and thoroughly verified, ensuring that all Increments work together. In order to provide value, the Increment must be usable.

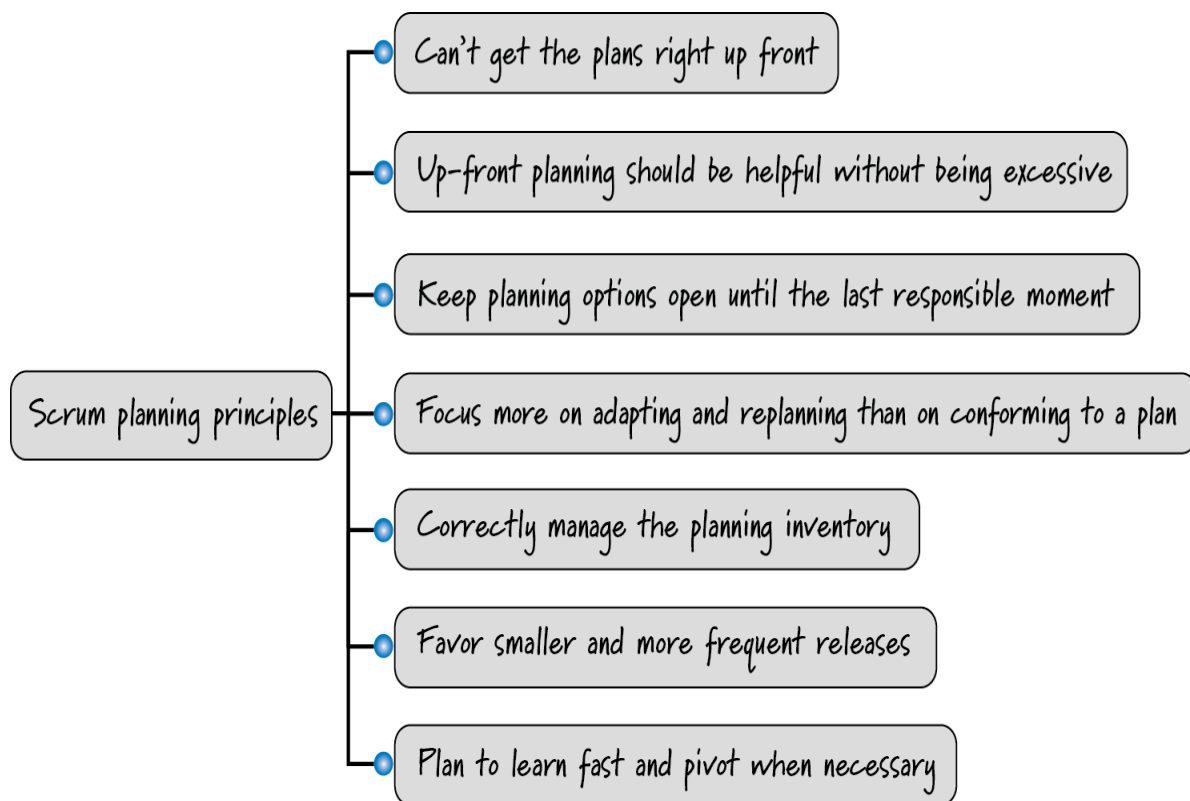
Multiple Increments may be created within a Sprint. The sum of the Increments is presented at the Sprint Review thus supporting empiricism. However, an Increment may be delivered to stakeholders prior to the end of the Sprint. The Sprint Review should never be considered a gate to releasing value.

Work cannot be considered part of an Increment unless it meets the Definition of Done (DoD).



Scrum Planning Principles

SCRUM PLANNING PRINCIPLES: AN OVERVIEW



Copyright © 2012, Kenneth S. Rubin and Innolution, LLC. All Rights Reserved.

SCRUM TEAMS ASSUME WE CAN'T GET THE PLANS RIGHT UP FRONT

The Scrum approach to planning is true to its empirical roots of inspection and adaptation. When developing using Scrum, we don't believe we can get it right up front, so we don't try to produce all of the planning artifacts up front. We do, however produce some of the planning artifacts early on to achieve a good balance between upfront and just-in-time planning.

SCRUM TEAMS KNOW THAT UPFRONT PLANNING SHOULD BE HELPFUL, NOT EXCESSIVE

Most of us have been involved in developing products where the level of detail associated with upfront predictive planning was just absurd. Does that mean we should do no upfront planning? No. That would be negligent and foolhardy. Equally foolhardy, however, is to plan to the point where it is difficult or costly to react to reality.

SCRUM TEAMS KEEP PLANNING OPTIONS OPEN UNTIL THE LAST RESPONSIBLE MOMENT

To achieve a good balance between upfront and just-in-time planning we are guided by the principle that we should keep important options open until the last responsible moment. This means we save the planning that is best performed in a just-in-time fashion for a time when we have much better information. Scrum teams shouldn't make early planning decisions based on

poor information.

SCRUM TEAMS FOCUS MORE ON ADAPTING AND REPLANNING THAN ON CONFORMING TO A PLAN

Many product development efforts place too much emphasis on upfront planning and not enough on continuous planning. The problem with this approach is that the more time we spend up front developing a highly predictive plan, the more likely we are to conform to the plan instead of updating it to respond to change. Scrum planning asserts that you can't get plans right upfront and you cannot eliminate change, no matter how hard you try. As such, Scrum teams value responding to change and replanning over following the upfront plan.

SCRUM TEAMS CORRECTLY MANAGE THE PLANNING INVENTORY

Scrum teams need to remember that creating a large inventory of predictive, not-yet-validated planning artifacts (e.g., Gantt charts) is potentially very wasteful.

Scrum teams validate assumptions by acquiring knowledge. As they do, they learn where reality differs from their original upfront plans. If they have a large volume of artifacts based on those upfront plans, they will be saddled with the waste of having to unwind and redo the future plans that have been invalidated by what they just learned.

This generates at least three forms of waste

- Wasted effort to produce the parts of the upfront plan that have to be discarded.
- Significant waste of having to update the upfront plan.
- Wasted opportunity of not having invested time in more valuable activities (such as delivering high-value working software) because that time was spent doing upfront work.

SCRUM TEAMS PLAN FOR SMALLER AND MORE FREQUENT RELEASES

Scrum favors smaller, more frequent releases (versus a single, large release) because frequent releases provide faster feedback and improve a product's return on investment (ROI). We can almost always improve the lifecycle profits of our product by leveraging incremental development and multiple releases of smaller marketable subsets of features.

SCRUM TEAMS PLAN TO LEARN FAST AND PIVOT WHEN NECESSARY

There is no amount of upfront predicting or guessing that will replace doing something, learning fast, and then pivoting when necessary. By pivoting, I mean changing directions while staying grounded in what we've learned. We need to be prepared to pivot quickly when we learn that our current plan is no longer valid.

Our goal is to move through the learning loop quickly and economically. So we should structure our plans with learning as a key goal. By getting fast feedback we can determine whether our plans are taking us in a viable direction. If not, we can pivot or redirect ourselves.

Release Planning in Scrum

Every agile organization must determine its own [cadence](#) for releasing features to its customers. Some choose to release every sprint. Others group the results of multiple sprints into one release. Still others release as soon as each feature is completed, a practice often referred to as **continuous deployment** or **continuous delivery**.

Whichever cadence an organization chooses, most find some amount of longer-term, release planning to be useful.

If the term release planning doesn't fit well with your organization's practices, replace the terminology with longer-term planning or milestone-driven planning. Whatever you choose to call it, **release planning in Scrum** targets a future state when important variables such as date, scope, and budget need to be balanced. The basic timing, participants, and process remain the same regardless of the name.

Release Planning in Scrum Overview:

Timing & Purpose

Release planning in Scrum happens every sprint, either as part of the sprint review or in the normal course of preparing for the subsequent sprint.

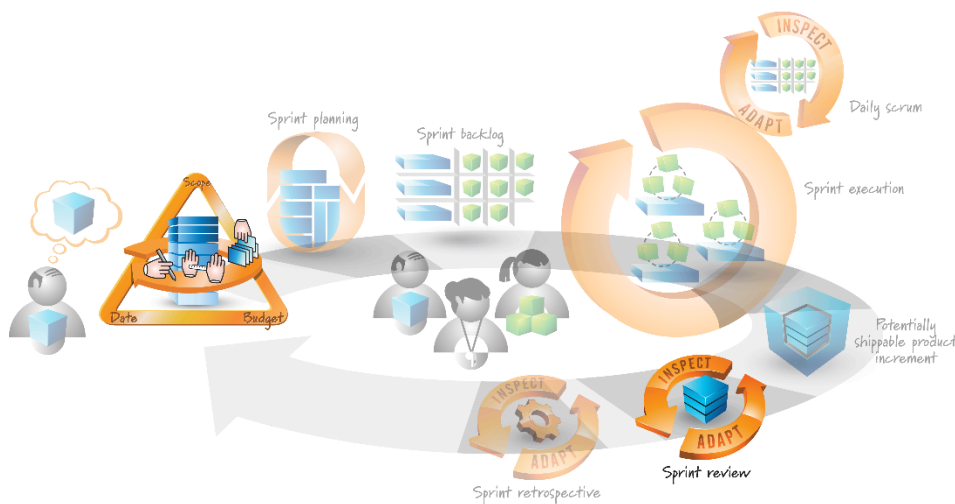
Initial release planning takes place following and typically lasts a day or two. The initial release plan will not be too complete or too precise because agile organizations understand that the release plan will be updated with the validated learning every sprint.

Participants

Release planning in Scrum involves collaboration between the stakeholders and the full Scrum team. Each person's exact involvement may vary over time.

Process

The image shown below illustrates release planning in Scrum.



Copyright © 2012, Kenneth S. Rubin and Innolution, LLC. All Rights Reserved.

The inputs for release planning include the Scrum development team’s velocity range as well as the outputs of envisioning: product vision, high-level product backlog, and optionally the product roadmap. The velocity can either be a forecast (in the case of a new team) or historical (in the case of an existing team). See “[Estimation and Velocity in Scrum: Chapter 7](#)” for more on velocity metrics.

The outputs of release planning, which are detailed later in the chapter, are collectively known as the **release plan**.

Release planning in Scrum consists of several activities:

- Review and update the release constraints of scope, date, and budget.
- Product backlog grooming.
- Review and update the minimum releasable features (MRFs)
- Product a sprint map (optional)

The activities can take place at multiple points in time, including immediately after product planning, as part of the initial-release planning activity, and during each sprint.

Constraints

The goal of Scrum release planning is to determine the most valuable next release and the desired level of quality. The release constraints of scope, date, and budget are important variables that affect how organizations achieve that goal. One or more of these constraints will probably be established as fixed during envisioning, with the remaining constraint(s) left flexible. For example, a fixed-date release has a set release date that must be met; the scope (and possibly the budget), however, are flexible.

Fixed Everything

Plan-driven predictive development approaches assumed that all three constraints could and should be fixed. These approaches assumed that the upfront plan could account for all requirements and as such, the scope would not change. With the scope fixed, it was then relatively easy to estimate a fixed cost and schedule. The problem is these “fixed-everything” upfront plans are almost always wrong. That’s why Scrum release planning requires that at least one of the variables (scope, budget, date) be flexible.

Fixed Scope and Date

Another suboptimal approach to Scrum release planning is to fix both the scope and date, but let the budget

be flexible. This approach suffers from a number of issues:

- It's difficult to increase budgets once development has begun.
- It's almost impossible to lock down these two variables at once. When push comes to shove, one of them will give.
- It assumes that applying more resources will help teams accomplish more in less time. This is only infrequently true, and even then, only up until a point.

Fixed Scope

A fixed-scope model to Scrum release planning is appropriate when the feature set is truly more important than the schedule. In this model, the release date is moved out until the full scope is complete. Because the end date is a moving target, budget can't really be fixed—People expect to be paid for their work!

One problem with a fixed scope approach is that in an organization with multiple teams or departments working on one project, a moving date makes coordination extremely challenging.

Another problem is that a fixed-scope approach is often an indication that the scope of the release is too large. A better solution might be to release smaller feature sets more often.

Fixed Date

A fixed date approach to Scrum release planning, in which the date and budget are fixed but the scope is flexible, is the approach most closely aligned with Scrum principles. Because the highest priority features are being developed and completed first, any features that don't make it into the first release are less valuable, which makes it easier to release the product as scheduled. A fixed-date approach also aligns well with Scrum's emphasis on timeboxing—prioritization is essential to making the model work.

The fixed-date model is even easier to use if the set of [minimum releasable features](#) (MRFs) is truly small. That means that any features that are beyond the scope of the MRFs are only nice-to-have, not must-have, features.

STARTING SCRUM

The first step in Scrum is for the Product Owner to articulate the product vision. Eventually, this evolves into a refined and prioritized list of features called the **Product Backlog**. This backlog exists (and evolves) over the lifetime of the product; it is the product road map (**Figure 2**). At any point, the Product Backlog is the single, definitive view of “everything that could be done by the Team ever, in order of priority”. Only a single Product Backlog exists; this means the Product Owner is required to make prioritization decisions across the entire spectrum, representing the interest of stakeholders and influenced by the team.

					New Estimates of Effort					
					Remaining as of Sprint...					
Item	Details (wiki URL)	Priority	Estimate of Value	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart (see UI sketches on wiki page)	...	1	7	5						
As a buyer, I want to remove a book in a shopping cart	...	2	6	7						
Improve transaction processing performance (see target performance metrics on wiki)	...	3	6	13						
Investigate solutions for speeding up credit card validation (see target performance metrics on wiki)	...	4	6	20						
Upgrade all servers to Apache 2.2.3	...	5	5	13						
Diagnose and fix the order processing script errors (bugzilla ID 100000)	...	6	7	7						

FIGURE 2. THE PRODUCT BACKLOG

The Product Backlog includes a variety of **items**, primarily new customer features (“enable all users to place book in shopping cart”), but also engineering improvement goals (“rework the transaction processing module to make it scalable”), exploratory or research work (“investigate solutions for speeding up credit card validation”), and, possibly, known defects (“diagnose and fix the order processing script errors”), if there are only a few problems. (A system with many defects usually has a separate defect tracking system.) The Product Backlog can be articulated in any way that is clear and sustainable, though either Use Cases or “user stories” are often used to describe the Product Backlog items in terms of their value to the end user of the product.

The subset of the Product Backlog that is intended for the current release is known as the **Release Backlog**, and in general, this portion is the primary focus of the Product Owner.

The Product Backlog is continuously updated by the Product Owner to reflect changes in the needs of the customer, new ideas or insights, moves by the competition, technical hurdles that appear, and so forth. The Team provides the Product Owner with estimates of the effort required for each item on the Product Backlog. In addition, the Product Owner is responsible for assigning a *business value estimate* to each individual item. This is usually an unfamiliar practice for a Product Owner. As such, it is something a Scrum Master may help the Product Owner learn to do. With these two estimates (effort and value) and perhaps with additional risk estimates, the Product Owner prioritizes the backlog (actually, usually just the Release Backlog subset) to maximize ROI (choosing items of high value with low effort) or secondarily, to reduce some major risk. As will be seen, these effort and value estimates may be refreshed each Sprint as people learn; consequently, this is a continuous re-prioritization activity as the Product Backlog is ever-evolving.

Scrum does not define techniques for expressing or prioritizing items in the Product Backlog and it does not define an estimation technique. A common technique is to estimate in terms of relative size (factoring in effort, complexity, and uncertainty) using a unit of “story points” or simply “points”.

Over time, a Team tracks how much work it can do each Sprint; for example, averaging 26 points per Sprint. With this information they can project a release date to complete all features, or how many features can be completed by a fixed date, if the average continues and nothing changes. This average is called the “velocity” of the team. Velocity is expressed in the same units as the Product Backlog item size estimates.

The items in the Product Backlog can vary significantly in size or effort. Larger ones are broken into smaller items during the Product Backlog Refinement workshop or the Sprint Planning Meeting, and smaller ones may be consolidated. The Product Backlog items for the upcoming next several Sprints should be small and fine-grained enough that they are understood by the Team, enabling commitments made in the Sprint Planning meeting to be meaningful; this is called an “actionable” size.

One of the myths about Scrum is that it prevents you from writing detailed specifications; in reality, it is up to the Product Owner and Team to decide how much detail is required, and this will vary from one backlog item to the next, depending on the insight of the Team, and other factors. State what is important in the least amount of space necessary – in other words, do not describe every possible detail of an item, just make clear what is necessary for it to be understood. Low priority items, far from being implemented and usually “coarse grained” or large, have less requirements details. High priority and fine-grained items that will soon be implemented tend to have more detail.

SPRINT PLANNING

At the beginning of each Sprint, the **Sprint Planning Meeting** takes place. It is divided into two distinct sub-meetings, the first of which is called **Sprint Planning Part One**.

In **Sprint Planning Part One**, the Product Owner and Team (with facilitation from the Scrum Master) review the high-priority items in the Product Backlog that the Product Owner is interested in implementing this Sprint. They discuss the goals and context for these high-priority items on the Product Backlog, providing the Team with insight into the Product Owner's thinking. The Product Owner and Team also review the "Definition of Done" (which was established earlier) that all items must meet, such as, "Done means coded to standards, reviewed, implemented with unit test-driven development (TDD), tested with 100 percent test automation, integrated, and documented." Part One focuses on understanding *what* the Product Owner wants.

Sprint Planning Part Two focuses on detailed task planning for *how* to implement the items that the Team decides to take on. The Team selects the items from the Product Backlog they commit to complete by the end of the Sprint, starting at the top of the Product Backlog (in other words, starting with the items that are the highest priority for the Product Owner) and working down the list in order. This is a key practice in Scrum: The Team decides how much work it will commit to complete, rather than having it assigned to them by the Product Owner. This makes for a more reliable commitment because the Team is making it based on its own analysis and planning, rather than having it decided by someone else. While the Product Owner does not have control over how much the Team commits to, he or she knows that the items the Team is committing to are drawn from the top of the Product Backlog – in other words, the items that he or she has rated as most important. The Team has the ability to lobby for items from further down the list; this usually happens when the Team and Product Owner realize that something of lower priority fits easily and appropriately with the high priority items.

The Sprint Planning Meeting will often last a number of hours, but no more than eight hours for a four-week Sprint – the Team is making a serious commitment to complete the work, and this commitment requires careful thought to be successful. The Team will probably begin the Sprint Planning Part Two by estimating how much time each member has for Sprint-related work – in other words, their average workday minus the time they spend attending meetings, doing email, taking lunch breaks, and so on. For most people this works out to 4-6 hours of time per day available for Sprint-related work. This is the team's capacity for the upcoming Sprint. See Figure

3.

FIGURE 3. ESTIMATING AVAILABLE HOURS

Sprint Length	2 weeks
----------------------	---------

Workdays during Sprint	8 days
-------------------------------	--------

Team Member	Available Days During Sprint*	Available Hours per Day	Total Available Hours
Tracy	8	4	32
Sanjay	7	5	35
Phillip	8	4	32
Jing	6	5	30

* Net of vacation and other days out of office

Once the capacity is determined, the Team figures out how many Product Backlog items they can complete in that time, and how they will go about completing them. This often starts with a design discussion at a whiteboard. Once the overall design is understood, the Team decomposes the Product Backlog items into work. The Team starts with the first item on the Product Backlog – in other words, the Product Owner’s highest priority item – and working together, breaks it down into individual tasks, which are recorded in a document called the **Sprint Backlog** (Figure 4).

As mentioned, the Product Owner must be available during Part Two (such as via the phone) so that clarification is possible. The Team will move sequentially down the Product Backlog in this way, until it’s used up all its estimated capacity. At the end of the meeting, the Team will have produced a list of all the tasks with estimates (typically in hours or fractions of a day).

Scrum encourages multi-skilled workers, rather than only “working to job title” such as a “tester” only doing testing. In other words, Team members “go to where the work is” and help out as possible. If there are many testing tasks, then *all* Team members may help. This does not imply that everyone is a generalist; no doubt some people are especially skilled in testing (and so on) but Team members work together and learn new skills from each other. Consequently, during task generation and estimation in Sprint Planning, it is not necessary – nor appropriate – for people to volunteer for all the tasks “they can do best.” Rather, it is better to only volunteer for one task at a time, when it is time to pick up a new task, and to consider choosing tasks that will on purpose involve learning (perhaps by pair work with a specialist).

All that said, there are *rare* times when *John* may do a particular task because it would take far too long or be impossible for others to learn – perhaps John is the only person with any artistic skill to draw pictures. Other Team members could not draw a “stick man” if their life depended on it. In this rare case – and if it is not rare and not getting rarer as the Team learns, there is something wrong – it may be necessary to ask if the total planned drawing tasks that *must* be done by John are feasible within the short Sprint.

Many Teams also make use of a visual task-tracking tool, in the form of a wall-sized task board where tasks (written on Post-It Notes) migrate during the Sprint across columns labeled “Not Yet Started,” “In Progress,” and “Completed.” See Figure 5.

New Estimates of Effort Remaining as of Day...									
Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database		5						
	create webpage (UI)		8						
	create webpage (Javascript logic)		13						
	write automated acceptance tests		13						
	update buyer help webpage		3						
	...								
Improve transaction processing performance	merge DCP code and complete layer-level tests		5						
	complete machine order for pRank		8						
	change DCP and reader to use pRank http API		13						

FIGURE 4. SPRINT BACKLOG

One of the pillars of Scrum is that once the Team makes its commitment, any additions or changes must be deferred until the next Sprint. This means that if halfway through the Sprint the Product Owner decides there is a new item he or she would like the Team to work on, he cannot make the change until the start of the next Sprint. If an external circumstance appears that significantly changes priorities, and means the Team would be wasting its time if it continued working, the Product Owner or the Team can terminate the Sprint. The Team stops, and a new Sprint Planning meeting initiates a new Sprint. The disruption of doing this is usually great; this serves as a disincentive for the Product Owner or Team to resort to this dramatic decision.

There is a powerful, positive influence that comes from the Team being protected from changing goals during the Sprint. First, the Team gets to work knowing with absolute certainty that its commitments will not change, that reinforces the Team's focus on ensuring completion. Second, it disciplines the Product Owner into really thinking through the items he or she prioritizes on the Product Backlog and offers to the Team for the Sprint.

By following these Scrum rules the Product Owner gains two things. First, he or she has the confidence of knowing the Team has made a commitment to complete a realistic and clear set of work it has chosen. Over time a Team can become quite skilled at choosing and delivering on a realistic commitment. Second, the Product Owner gets to make whatever changes he or she likes to the Product Backlog before the start of the *next* Sprint. At that point, additions, deletions, modifications, and re-prioritizations are all possible and acceptable. While the Product Owner is not able to make changes to the selected items under development during the current Sprint, he or she is only one Sprint's duration or less away from making any changes they wish. Gone is the stigma around change – change of direction, change of requirements, or just plain changing your mind – and it may be for this reason that Product Owners are usually as enthusiastic about Scrum as anyone.

DAILY SCRUM

Once the Sprint has started, the Team engages in another of the key Scrum practices: The **Daily Scrum**. This is a short (15 minutes or less) meeting that happens every workday at an appointed time. Everyone on the Team attends. To keep it brief, it is recommended that everyone remain standing. It is the Team's opportunity to synchronize their work and report to each other on obstacles. In the Daily Scrum, one by one, each member of the Team reports three (and only three) things *to the other members of the Team*: (1) What they were able to get done since the last meeting; (2) what they are planning to finish by the next meeting; and (3) any blocks or impediments that are in their way.

The Daily Scrum is not a status meeting to report to a manager; it is a time for a self-organizing Team to share with each other what is going on, to help them coordinate. Someone makes note of the blocks, and the Scrum Master is responsible to help Team members resolve them. There is no discussion during the Daily Scrum, only reporting answers to the three questions; if discussion is required it takes place immediately after the

Daily Scrum in a follow-up meeting, although in Scrum no one is required to attend this. This follow-up meeting is a common event where the Team adapts to the information they heard in the Daily Scrum: in other words, another inspect and adapt cycle. It is generally recommended *not* to have managers or others in positions of perceived authority attend the Daily Scrum. This risks making the Team feel “monitored” – under pressure to report major progress every day (an unrealistic expectation), and inhibited about reporting problems – and it tends to undermine the Team’s self-management, and invite micromanagement. It would be more useful for a stakeholder to instead reach out to the Team following the meeting, and offer to help with any blocks that are slowing the Team’s progress.

UPDATING SPRINT BACKLOG & SPRINT BURNDOWN CHART

The Team in Scrum is self-managing, and in order to do this successfully, it must know how it is doing. Every day, the Team members update their estimate of the amount of time remaining to complete their current task in the **Sprint Backlog** (Figure 6). Following this update, someone adds up the hours remaining for the Team as a whole, and plots it on the **Sprint Burndown Chart** (Figure 7). This graph shows, each day, a new estimate of how much work (measured in person hours) remains until the Team’s tasks are finished. Ideally, this is a *downward* sloping graph that is on a trajectory to reach “zero effort remaining” by the last day of the Sprint. Hence it is called a *burndown* chart. And while sometimes it looks good, often it does not; this is the reality of product development. The important thing is that it shows the Team their progress towards their goal, not in terms of how much time was *spent* in the past (an irrelevant fact in terms of *progress*), but in terms of how much work *remains in the future* – what separates the Team from their goal. If the burndown line is not tracking downwards towards completion near the end of the Sprint, then the Team needs to adjust, such as to reduce the scope of the work or to find a way to work more efficiently while still maintaining a sustainable pace.

While the Sprint Burndown chart can be created and displayed using a spreadsheet, many Teams find it is more effective to show it on paper on a wall in their workspace, with updates in pen; this “low-tech/high-touch” solution is fast, simple, and often more visible than a computer chart.

			New Estimates of Effort Remaining at end of Day...						
Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database	Sanjay	5	4	3	0	0	0	
	create webpage (UI)	Jing	3	3	3	2	0	0	
	create webpage (Javascript logic)	Tracy & Sam	2	2	2	2	1	0	
	write automated acceptance tests	Sarah	5	5	5	5	5	0	
	update buyer help webpage	Sanjay & Jing	3	3	3	3	3	0	
...									
Improve transaction processing performance	merge DCP code and complete layer-level tests		5	5	5	5	5	5	
	complete machine order for pRank		3	3	8	8	8	8	
	change DCP and reader to use pRank http API		5	5	5	5	5	5	
...									
		Total (person hours)	50	49	48	44	43	34	

FIGURE 6. DAILY UPDATES OF WORK REMAINING ON THE SPRINT BACKLOG

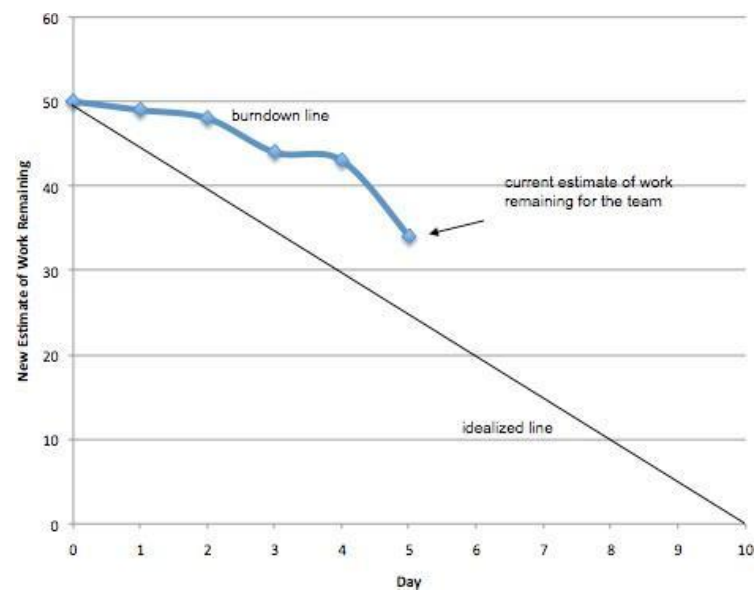


FIGURE 7. SPRINT BURNDOWN CHART

PRODUCT BACKLOG REFINEMENT

One of the lesser known, but valuable, guidelines in Scrum is that five or ten percent of each Sprint must be dedicated by the Team to refining (or “grooming”) the Product Backlog. This includes detailed requirements analysis, splitting large items into smaller ones, estimation of new items, and re-estimation of existing items. Scrum is silent on how this work is done, but a frequently used technique is a focused workshop near the end of the Sprint, so that the Team and Product Owner can dedicate themselves to this work without interruption.

For a two-week Sprint, five percent of the duration implies that each Sprint there is a half-day Product Backlog Refinement workshop. This refinement activity is not for items selected for the current Sprint; it is for items for the future, most likely in the next one or two Sprints. With this practice, Sprint Planning becomes relatively simple because the Product Owner and Scrum Team start the planning with a clear, well-analyzed and carefully estimated set of items. A sign that this refinement workshop is not being done (or not being done well) is that Sprint Planning involves significant questions, discovery, or confusion and feels incomplete; planning work then often spills over into the Sprint itself, which is typically not desirable.

ENDING THE SPRINT

One of the core tenets of Scrum is that the duration of the Sprint is never extended – it ends on the assigned date regardless of whether the Team has completed the work it committed to. A Team typically over-commits in its first few Sprints and fails to accomplish its commitments. Sometimes it then overcompensates and under-commits, and finishes early (in which case it can ask the Product Owner for more Product Backlog items to work on). But by the third or fourth Sprint a Team has typically figured out what it is capable of delivering (most of the time), and they will meet their Sprint goals more reliably after that. Teams are encouraged to pick one duration for their Sprints (say, two weeks) and not change it. This helps the Team learn how much it can accomplish, which helps in both estimation and longer-term release planning. It also helps the Team achieve a rhythm for their work; this is often referred to as the “heartbeat” of the Team in Scrum.

SPRINT REVIEW

After the Sprint ends, there is the **Sprint Review**, where the Team and the Product Owner review the Sprint. This is often mislabeled the “demo” but that does not capture the real intent of this meeting. A key idea in Scrum is *inspect and adapt*. To see and learn what is going on and then evolve based on feedback, in repeating cycles. The Sprint Review is an inspect and adapt

activity for the *product*. It is a time for the Product Owner to learn what is going on with the product and with the Team (that is, a review of the Sprint); and for the Team to learn what is going on with the Product Owner and the market. Consequently, the most important element of the Review is an in-depth *conversation* between the Team and Product Owner to learn the situation, to get advice, and so forth. The review includes a demo of what the Team built during the Sprint, but if the focus of the review is a demo rather than conversation, there is an imbalance.

A useful – but often overlooked – Scrum guideline is that it the Scrum Master’s responsibility to ensure that everyone knows the “Definition of Done” defined for this product or release. He prevents the team from demonstrating or discussing Product Backlog Items that are not ‘done’ according to the “Definition of Done.” Items that are not ‘done’ go back to the Product Backlog and will be re-prioritized by the Product Owner. In way, there is transparency regarding the quality of the work; Teams cannot fake the quality by presenting software that appears to work well, but may be implemented with a messy pile of poor quality and untested code.

Present at this meeting are the Product Owner, Team members, and Scrum Master, plus customers, stakeholders, experts, executives, and anyone else interested. The demo portion of the Sprint Review is not a “presentation” the Team gives – there is no slideware. A guideline in Scrum is that no more than 30 minutes should be spent preparing for the review, otherwise it suggests something is wrong with the work of the Team. It is simply a demo of what has been built. Anyone present is free to ask questions and give input.

SPRINT RETROSPECTIVE

The Sprint Review involves inspect and adapt regarding the *product*. The **Sprint Retrospective**, which follows the Review, involves inspect and adapt regarding the *process*. This is a practice that some Teams skip, and that’s unfortunate, because it’s the main mechanism for taking the visibility that Scrum provides into areas of potential improvement, and turning it into results. It’s an opportunity for the Team to discuss what’s working and what’s not working, and agree on changes to try. The Team and Scrum Master will attend, and the Product Owner is welcome but not required to attend. Sometimes the Scrum Master can act as an effective facilitator for the retrospective, but it may be better to find a neutral outsider to facilitate the meeting; a good approach is for Scrum Masters to facilitate each others’ retrospectives, which enables cross-pollination among Teams.

There are many techniques for conducting a Sprint Retrospective, and the book *Agile Retrospectives* (Derby, Larsen 2006) provides a useful catalogue of techniques. A simple way to structure the discussion is to draw two columns on a whiteboard, labeled “What’s Working Well” and “What Could Work

Better” – and then go around the room, with each person adding one or more items to either list. As items are repeated, check marks are added next to them, so the common items become clear. Then the Team looks for underlying causes, and agrees on a small number of changes to try in the upcoming Sprint, along with a commitment to review the results at the next Sprint Retrospective.

A useful practice at the end of the Retrospective is for the Team to label each of the items in each column with either a “C” if it is *caused* by Scrum (in other words, without Scrum it would not be happening), or an “E” if it is *exposed* by Scrum (in other words, it would be happening with or without Scrum, but Scrum makes it known to the Team), or a “U” if it’s unrelated to Scrum (like the weather). The Team may find a lot of C’s on the “What’s Working Well” side of the board, and a lot of E’s on the “What Could Work Better ”; this is good news, even if the “What Could Work Better” list is a long one, because the first step to solving underlying issues is making them visible, and Scrum is a powerful catalyst for that.

UPDATING RELEASE BACKLOG & BURNDOWN CHART

At this point, some items have been finished, some have been added, some have new estimates, and some have been dropped from the release goal. The Product Owner is responsible for ensuring that these changes are reflecting in the Release Backlog (and more broadly, the Product Backlog). In addition, Scrum includes a **Release Burndown** chart that shows progress towards the release date. It is analogous to the Sprint Burndown chart, but is at the higher level of items (requirements) rather than fine-grained tasks. Since a new Product Owner is unlikely to know why or how to create this chart, this is another opportunity for a Scrum Master to help the Product Owner. example of the Release Backlog and Release Burndown chart.

New Estimates of Effort Remaining at end of Sprint...										
Item	Details (wiki URL)	Priorty	Estimate of Value	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart (see UI sketches on wiki page)	...	1	7	5	0	0	0			
As a buyer, I want to remove a book in a shopping cart	...	2	6	2	0	0	0			
Improve transaction processing performance (see target performance metrics on wiki)	...	3	6	13	13	0	0			
Investigate solutions for speeding up credit card validation (see	...	4	6	20	20	20	0			

target performance metrics on wiki)										
Upgrade all servers to Apache 2.2.3 ...	...	5	5	13	13	13	13			
Diagnose and fix the order processing script errors (bugzilla ID 14823)	...	6	2	3	3	3	3			
As a shopper, I want to create and save a wish list	...	7	7	40	40	40	40			
As a shopper, I want to to add or delete items on my wish list	...	8	4	20	20	20	20			

... ..

Total 537 580 570 500

FIGURE 8. RELEASE BACKLOG (A SUBSET OF THE PRODUCT BACKLOG)

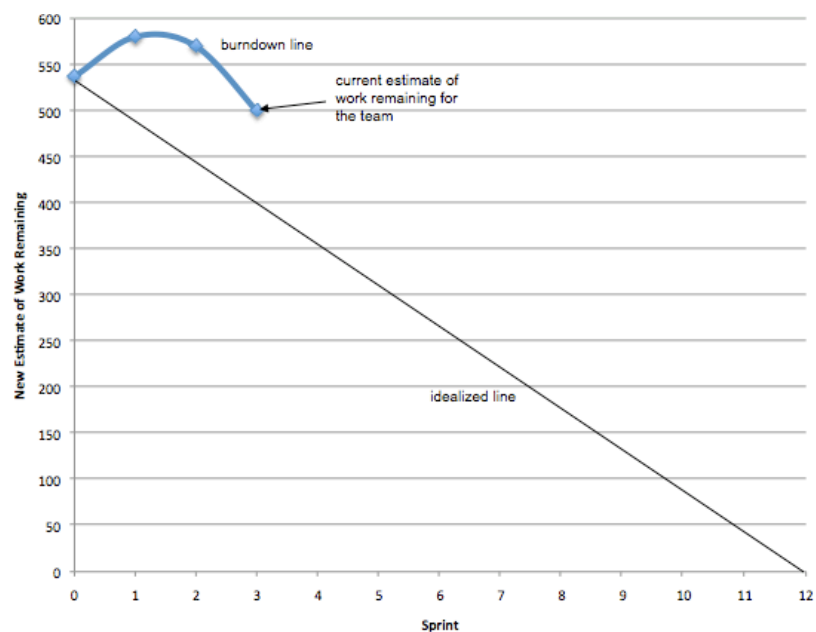


FIGURE 9. RELEASE BURNDOWN CHART

STARTING THE NEXT SPRINT

Following the Sprint Review, the Product Owner may update the Product Backlog with any new insight. At this point, the Product Owner and Team are ready to begin another Sprint cycle. There is no down time between Sprints – Teams normally go from a Sprint Retrospective one afternoon into the next Sprint Planning the following morning (or after the weekend).

One of the principles of agile development is “sustainable pace”, and only by working regular hours at a reasonable level can Teams continue this cycle indefinitely.

